

## CAHIER DES CHARGES POUR APPLI DE GESTION DE MOTS DE PASSE

### 1. IDEE PRINCIPALE

Une app web simple pour stocker ses identifiants en sécurité.

Disons que l'utilisateur se connecte une fois avec un mot de passe maître, puis accède à tous ses autres mots de passe

### 2. FONCTIONNALITÉS

#### 2.1 Authentification

S'inscrire : L'utilisateur doit fournir un email et choisir un mot de passe maître (au moins 12 caractères). C'est ce mot de passe qui va déverrouiller toute l'app.

Se connecter : L'utilisateur rentre son email et son mot de passe maître.

Se déconnecter : Bouton "Logout" qui termine la session.

Important : Le mot de passe est immédiatement chiffré avant d'être envoyé au serveur.

Utiliser des ORM pour les injections SQL

#### BONUS : Notification de connexion par email

- À chaque login réussi : email envoyé afin de détecter les connexions suspectes :  
Vous vous êtes connecté à 12 :43

#### 2.2 Gestion des identifiants

Ajouter un identifiant : L'utilisateur peut créer une nouvelle entrée en remplissant :

- Service (nom du site/app : Gmail, Netflix, etc.)
- Login ou Email
- Mot de passe

Voir ses identifiants : Affichage d'une liste avec le nom du service, l'email/login associé. Les mots de passe sont masqués par défaut (des points).

Afficher un password : Cliquer l'icône "œil" ou show pour révéler le password en clair

Copier un password : Bouton "Copy" qui met le password en presse-papiers.

Éditer un identifiant : Modifier le service, login, password ou notes. Les changements sont chiffrés et sauvegardés.

Supprimer : Suppression avec confirmation du type (êtes-vous sûr de vouloir supprimer)

## 2.3 Générateur de MDP

Générer un password automatiquement : Bouton "Generate Password" qui crée un password aléatoire sécurisé.

Options du générateur :

Longueur (par défaut 16 caractères, min 8, max 64)

Faire en sorte que le password soit un mix de lettres MAJ/MIN/SYMBOLLES (qu'il soit fort)

Bouton "Regenerate" pour relancer

Bouton "Use this password" pour l'insérer dans le formulaire

Le générateur affiche aussi la "force" du password généré (Faible - Moyen - Fort).

## 3. INTERFACE UTILISATEUR

Écrans principaux :

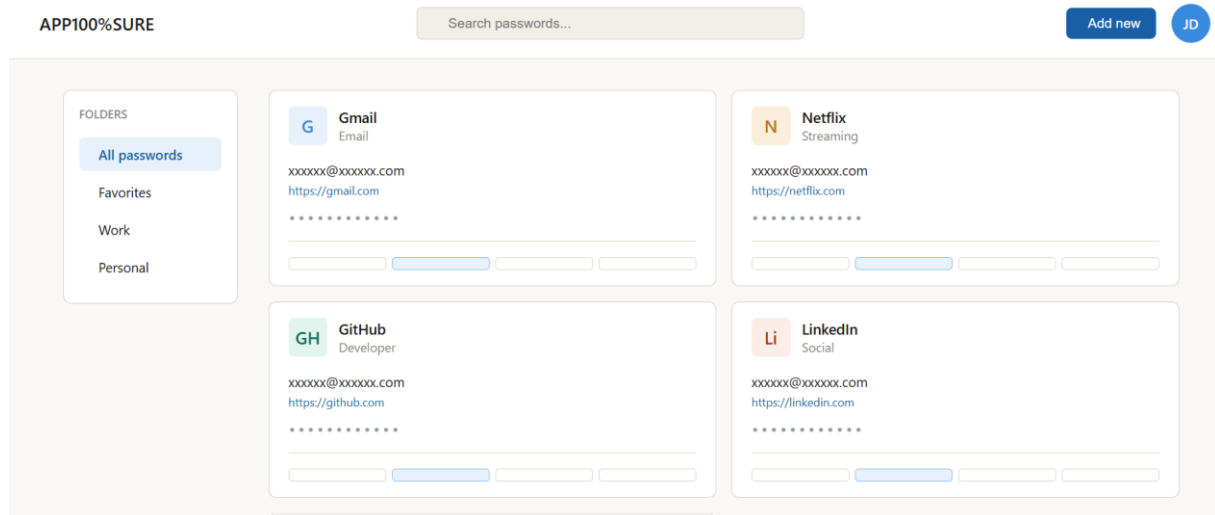
Écran de Login

- Logo et titre de l'app
- Champ Email
- Champ Master Password
- Bouton "Se connecter"
- Lien "Pas de compte ? S'inscrire"

Écran d'Inscription

- Champ Email
- Champ Master Password (avec indicateur de force tjours si possible)
- Confirmation Master Password
- Bouton "S'inscrire"
- Lien "Déjà inscrit ? Se connecter"

Dashboard / Écran Principal



Ceci est un exemple de ce à quoi peut ressembler l'interface de l'app.

- Logo **APP100%SURE (choisir votre propre nom)** avec icône cadenas
- Barre de recherche en temps réel pour filtrer par service ou login
- Bouton **"Add new"** pour ajouter un nouveau service
- Menu utilisateur (initiales, clic pour voir apparaître le bouton logout)
- URL de l'app concernée + mail et mdp

### Sidebar (Dossiers)

- All passwords (sélectionné par défaut)
- Favorites
- Work (BONUS)
- Personal (BONUS)

## 4. Sécurité basique

**Password chiffré :** Avant d'être envoyé au serveur, chaque password sauvegardé est chiffré (type de chiffrement au choix). La clé de chiffrement dérive du Master Password de l'utilisateur. Le serveur ne stocke que du chiffré

**Master password :** Le Master Password n'est jamais stocké en clair en base de données. On stocke un hash avec un salt unique par utilisateur. Ça permet de vérifier le password au login sans le connaître.

**Transmission :** Tous les échanges client-serveur se font en HTTPS . Pas d'HTTP.

**Session :** Après chaque login réussi, l'utilisateur reçoit un token JWT qui expire après 30 minutes d'inactivité. Si le token expire, l'utilisateur est redirigé vers le login.

**Validation des inputs :** Email validé, Master Password vérifié (au moins 12 caractères)

## 5. Architecture de base et technologie

FrontEnd : React (Libsodium pas obligé)

Backend : Node.js + Base de données

